



Introduction to Software Configuration Management Part I: The Team

Lars Bendix
bendix@sneSCM.org

*Specialist Network of Excellence Department of Computer Science
in Software Configuration Management Lund University
(sneSCM.org) Sweden*

https://fileadmin.cs.lth.se/cs/Personal/Lars_Bendix/Teaching/1-ECTS-SCM/



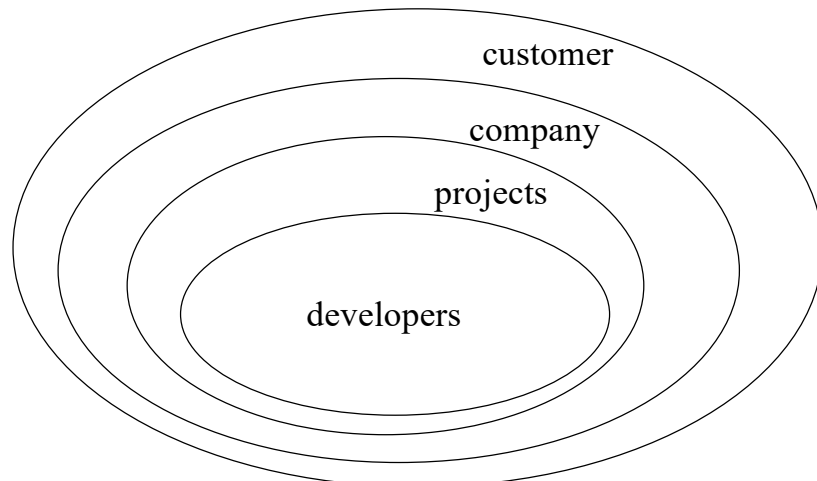
Learning goals

After this first part the student will:

- understand co-ordination problems
- understand some important versioning concepts
- understand basic co-ordination strategies
- know some basic CM concepts
- be able to use a version control tool



History of SCM



What is SCM?

Software Configuration Management:
is the discipline of organising, controlling and
managing the development and evolution of
software systems. (IEEE, ISO,...)

The goal is to maximize [programmer] productivity
by minimizing [co-ordination] mistakes. (Babich)



Problems



Identification:

You should be able to identify the single components and configurations.

- This worked yesterday, what has happened?
- Do we have the latest version?
- I have already fixed this problem. Why is it still there?

Change tracking:

Helps in tracking which changes have been made to which modules and by whom, when and why.

- Has this problem been fixed?
- Who is responsible for this change?
- This change looks obvious - has it been tried before?



Problems



Software production:

Construction of a program involves pre-processing, compilation, linking, etc.

- I just corrected this, has something not been compiled?
- How was this binary produced?
- Did we do all the necessary steps in the right order?

Concurrent updating:

The system should offer possibilities for concurrent changes to components.

- Why did my changes disappear?
- How do I get these changes into my version?
- Are our changes in conflict with each other?



Excuses for not using CM?



- CM only applies to source code
- CM is not appropriate during development because we use rapid prototyping
- It's not that big a project
- You can't stop people from making a quick patch
- We lower our cost by using only minimum-wage persons on our CM staff because CM does not require much skill ;-)



How does a programmer spend his time?



- 50% interacting with other team members
- 30% working alone
- 20% non-productive activities



(Software) development



- re-use things
- sharing things
- memory/history

- collaboration
- co-ordination
- communication



SCM Hall of Fame



Wayne Babich, 1986:

*Software Configuration Management –
Coordination for Team Productivity*

Team co-ordination problems:

- shared data
- double maintenance
- simultaneous update

“An ounce of [SBoM] is worth
a pound of analysis”



Wayne Babich I



Sometimes it is embarrassing to be a computer programmer. What other profession has such a remarkable rate of schedule and cost overrun and outright failure? [...]

Our failures are not of the individual contributors; most of us design, code and debug adequately or even well. Rather, the failure is one of coordination. Somehow we lack the ability to take 20 or 30 good programmers and meld them into a consistently productive team.

Wayne A. Babich, 1986



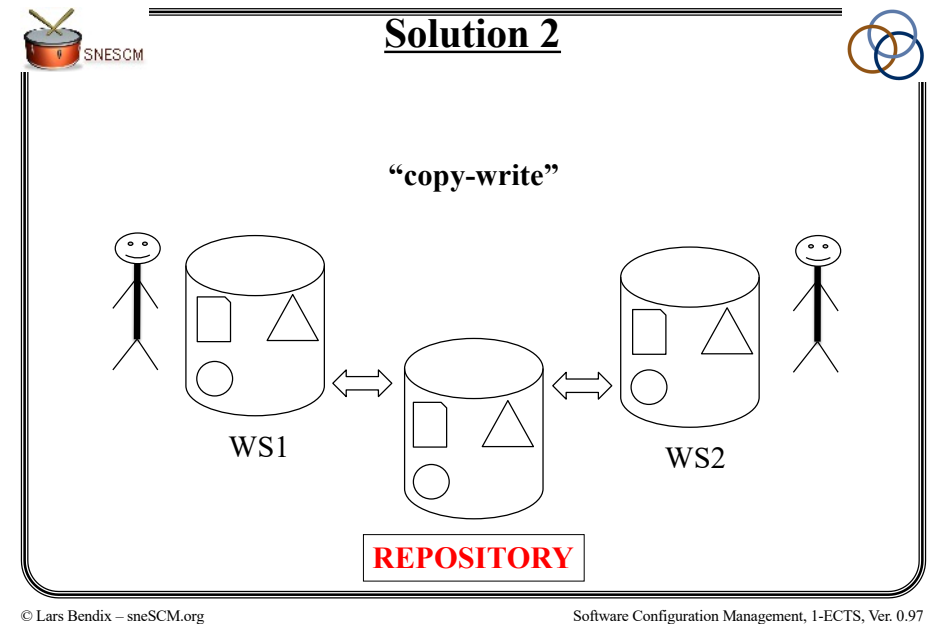
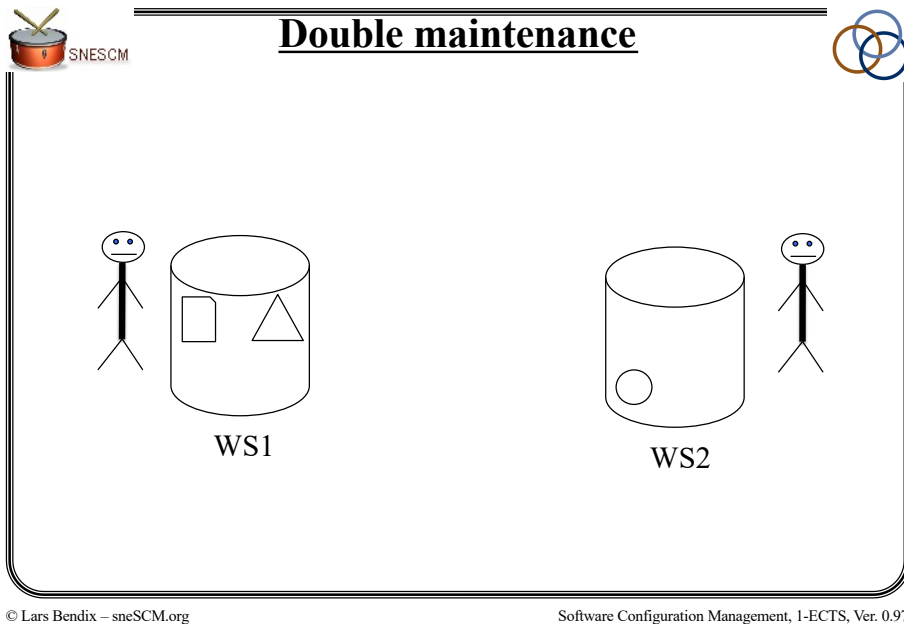
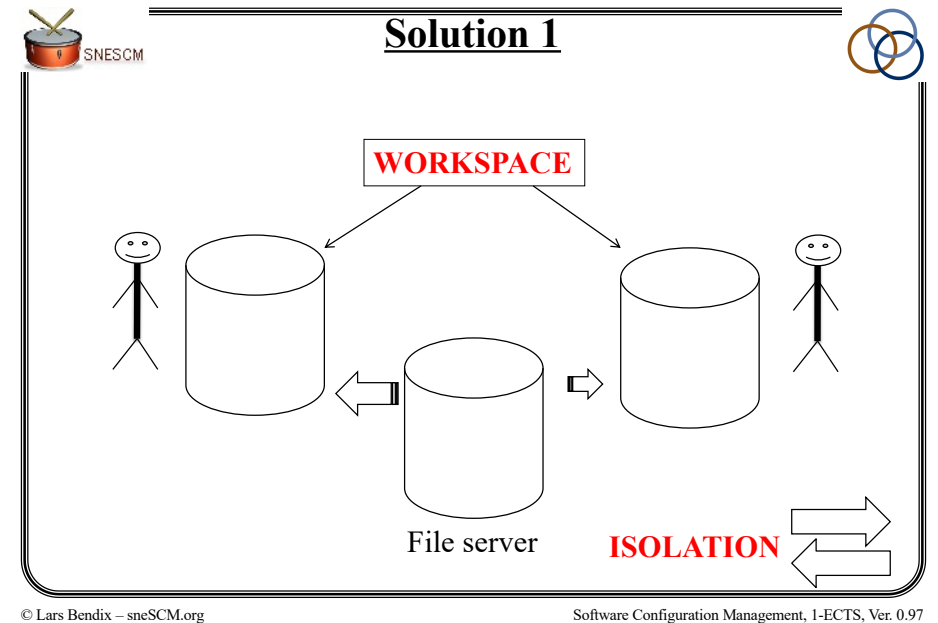
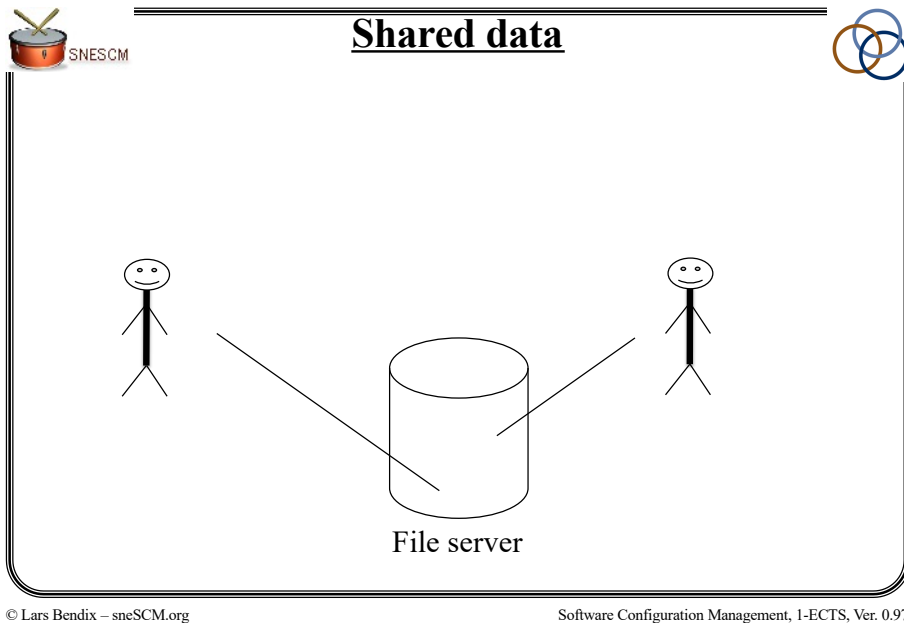
Wayne Babich II



The term configuration management derives from the hard engineering disciplines [...], which use change control techniques to manage blueprints and other design documents. The term software configuration management has traditionally been applied to the process of describing and tracking releases of software as the product leaves the development group for the outside world.

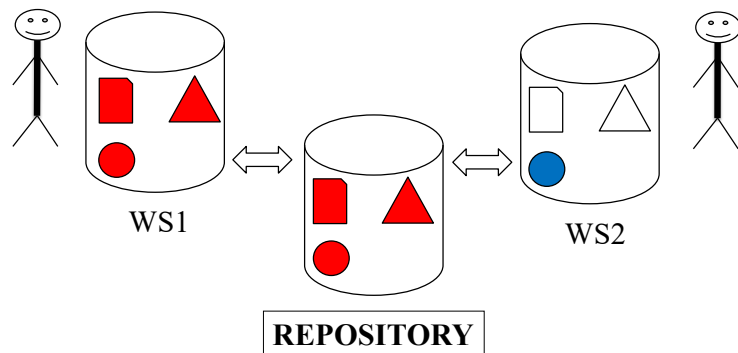
I use the term in a more expansive sense. I include not just the formal release of software to the customer, but the day-to-day and minute-by-minute evolution of the software inside the development team. Controlled evolution means that you not only understand *what* you have *when* you are **delivering it**, but you also understand *what* you have *while* you are **developing it**.

Wayne A. Babich, 1986





Simultaneous update



Co-ordination



Working in isolation:

- local dynamicity
- global stability
- problems:
 - double maintenance

Working in group:

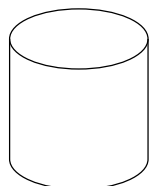
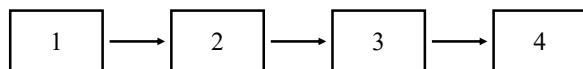
- global dynamicity
- problems:
 - shared data
 - simultaneous update



Solution 3



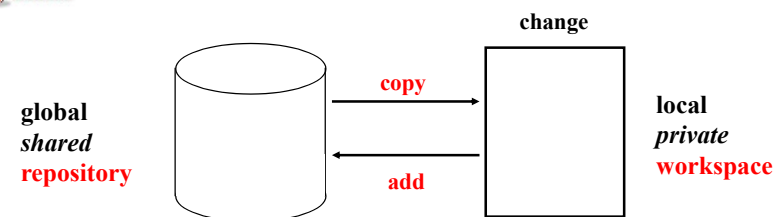
“copy-write” => “copy-add”



VERSIONING



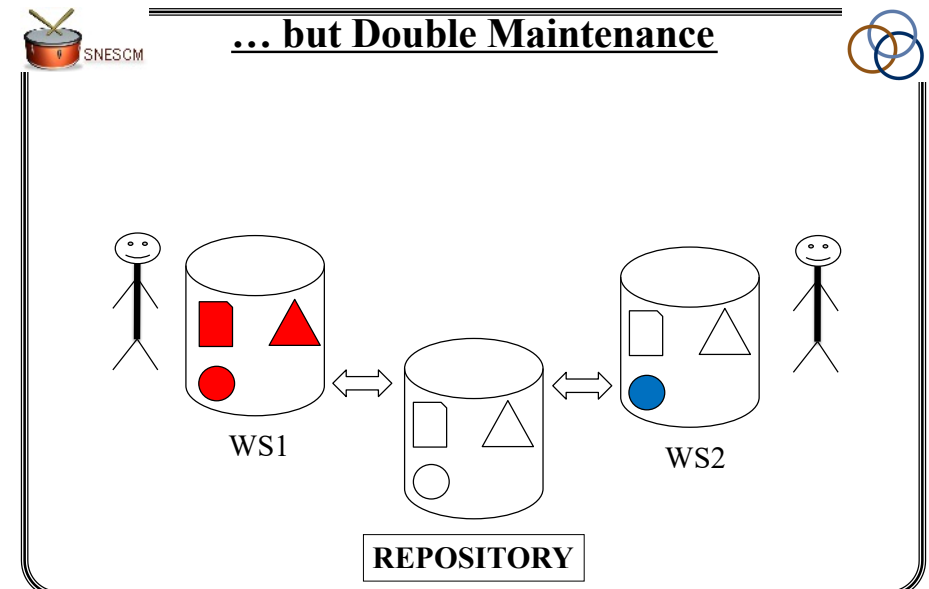
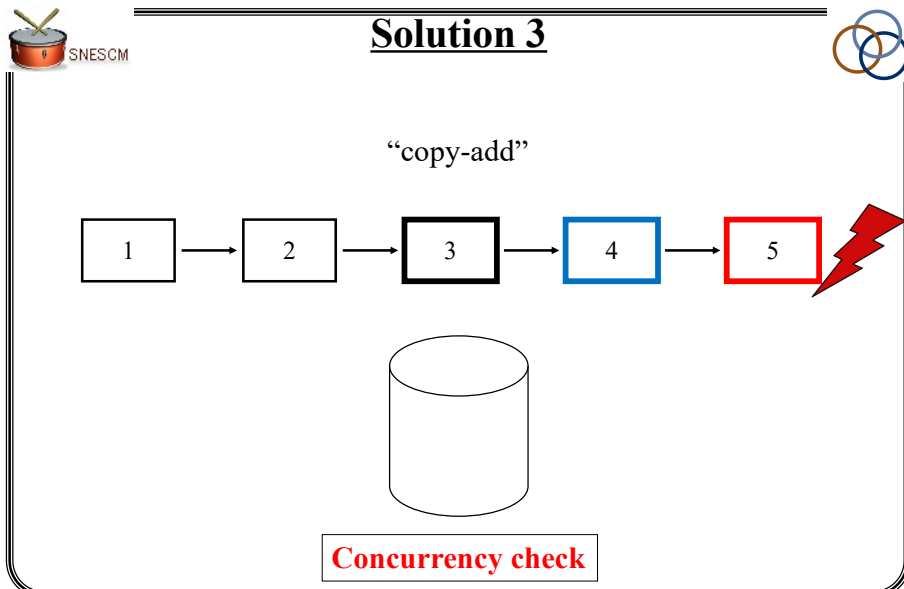
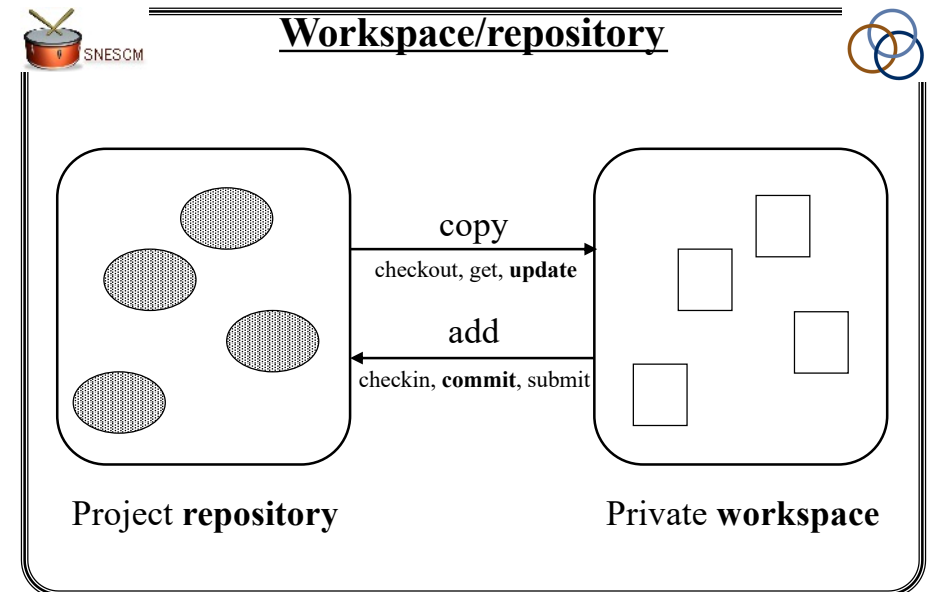
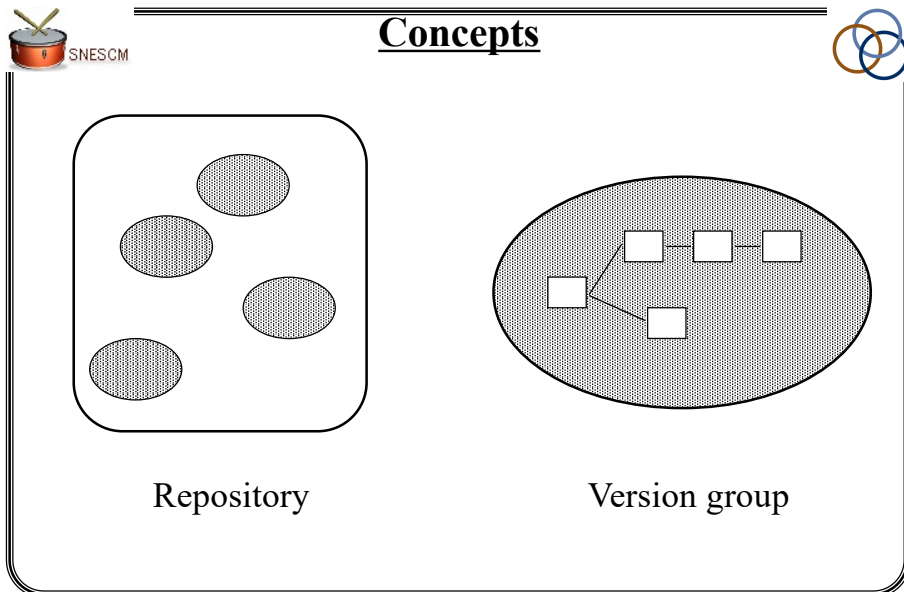
Model



Principle: components are **immutable**

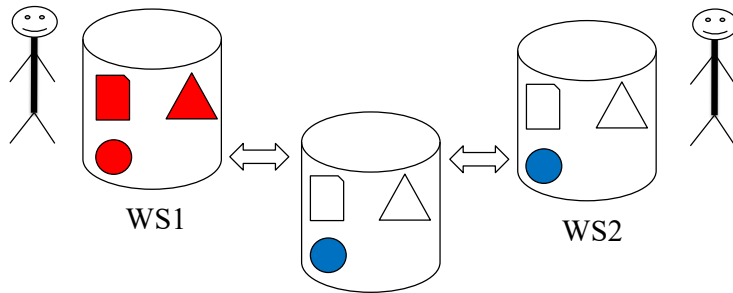
Babich:

- shared data
- double maintenance
- simultaneous update





... but Double Maintenance



NOW WHAT???



Parallel work

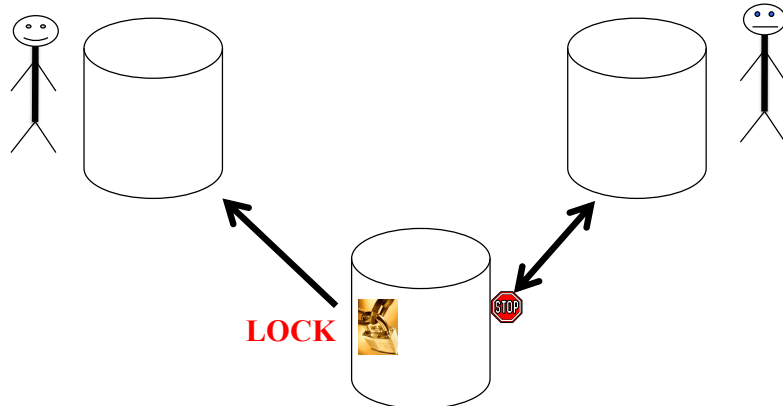


Concurrency strategies:

- pessimistic (**locking**)
- optimistic (**copy-merge**)



Locking



Copy/merge work model



Can we *lock* the things we want to work on? NO!

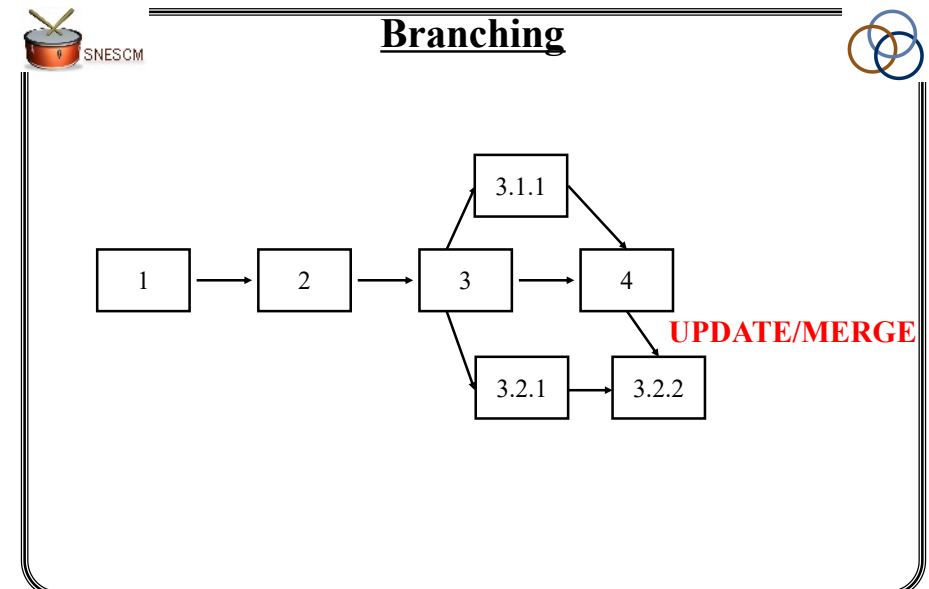
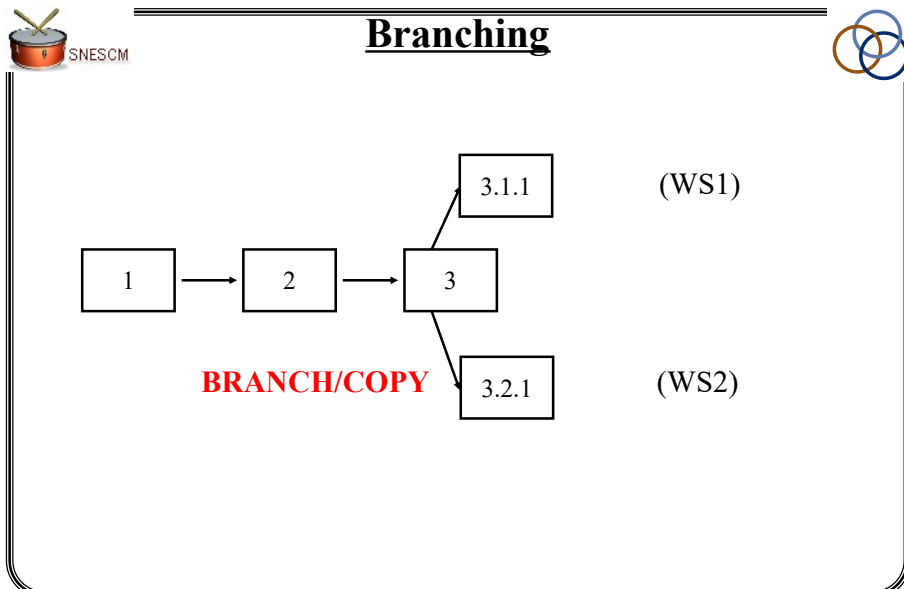
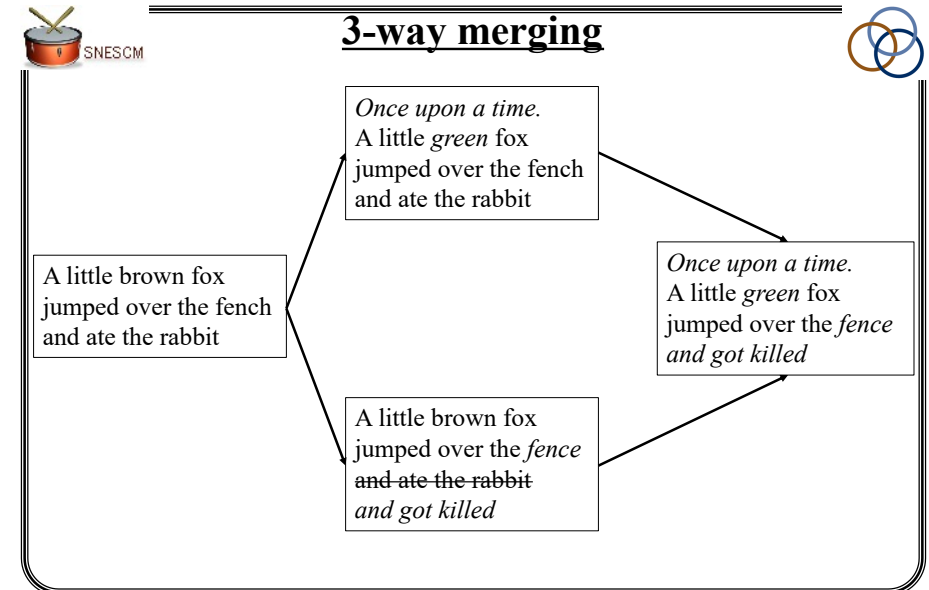
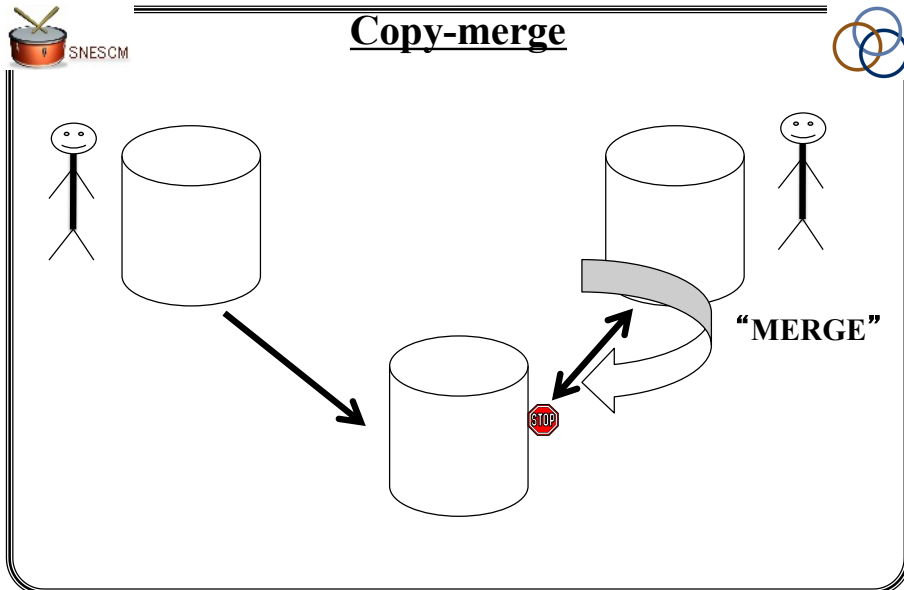
So we **copy** everything to our workspace ...

... and everyone else copy to their workspaces ...

⇒ double maintenance !!

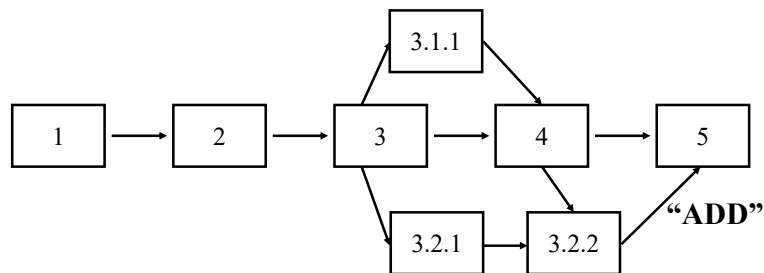
Fortunately "update" has a built-in **merge** facility:

- We get stopped from adding by the "concurrency check"
- We then merge *from* the repository *into* the workspace
- Then we check and fix possible problems
- Finally we commit (**add safely**) to the repository





Branching



Parallel work II

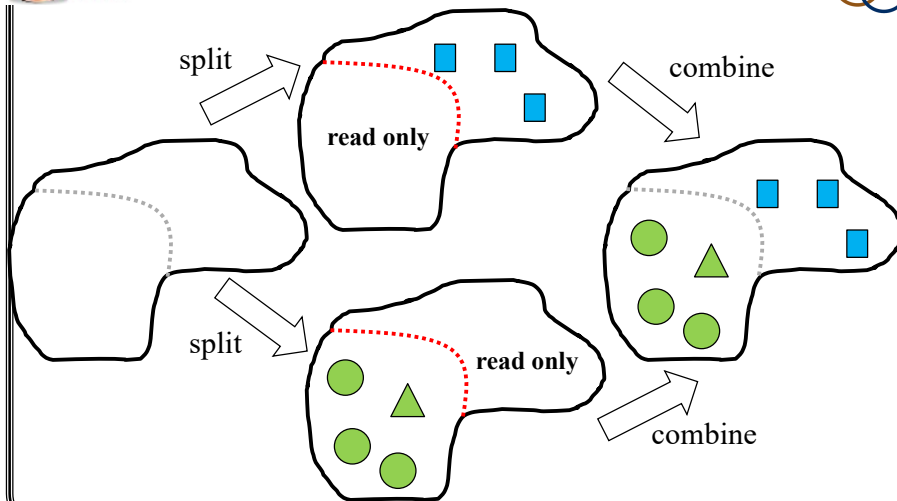


Concurrency strategies:

- *pessimistic (locking)*
- *optimistic (copy-merge)*
- mixed (**split-combine**)



Split-combine



Long transactions



We do our change as a **logical unit**, we want this logical unit to be kept/recorded:

⇒ if there is anything that "conflicts", we must abort the **whole** commit (**long transaction**) also known as the "*all or nothing*" principle

So far we have looked only at one type of "conflicts":

- people changing the same file(s) in parallel
- what about people changing *different* files?



Strict long transactions



We do our change on a **system**, we test that it works and want the state of our workspace to become the latest and greatest (LAG) in the repository:

- ⇒ I guarantee that the LAG (my workspace) is of perfect quality (and has all) – and put my name to it
- ⇒ if there is anything that "conflicts" (someone has changed *the system*), we must abort the **whole** commit (**strict long transaction**)
- ⇒ if we allow the commit to go through, it will be a "structural merge" into the repository – it will create a *new* configuration that has *not* been tested



Strict long transactions



We are interested in whole **systems/configurations**:

- should we merge systems/configurations?
- yes, but **never** into the repository!

From *physical* conflicts to *logical* conflicts:

No *new* configurations should be created in the repo
New configurations should be created in the *workspace*
New configurations should be *tested* in the workspace
Then they should be "*copied/added*" to the repository

What can we do if the tool does not have SLT?



Working together



I might still not want things to change:

- baseline (freezing a configuration)

I might want to know who is affected by my changes:

- traceability

I might want to know exactly how a system was built:

- software bill of material (SBoM)